

CREWAI // LAB

CREWAI

Manual del alumno

MULTI-AGENTES · LABORATORIO PRÁCTICO

DE UNA IA SOLA → A UN EQUIPO

Tu misión: construirlo, ejecutarlo y entenderlo.

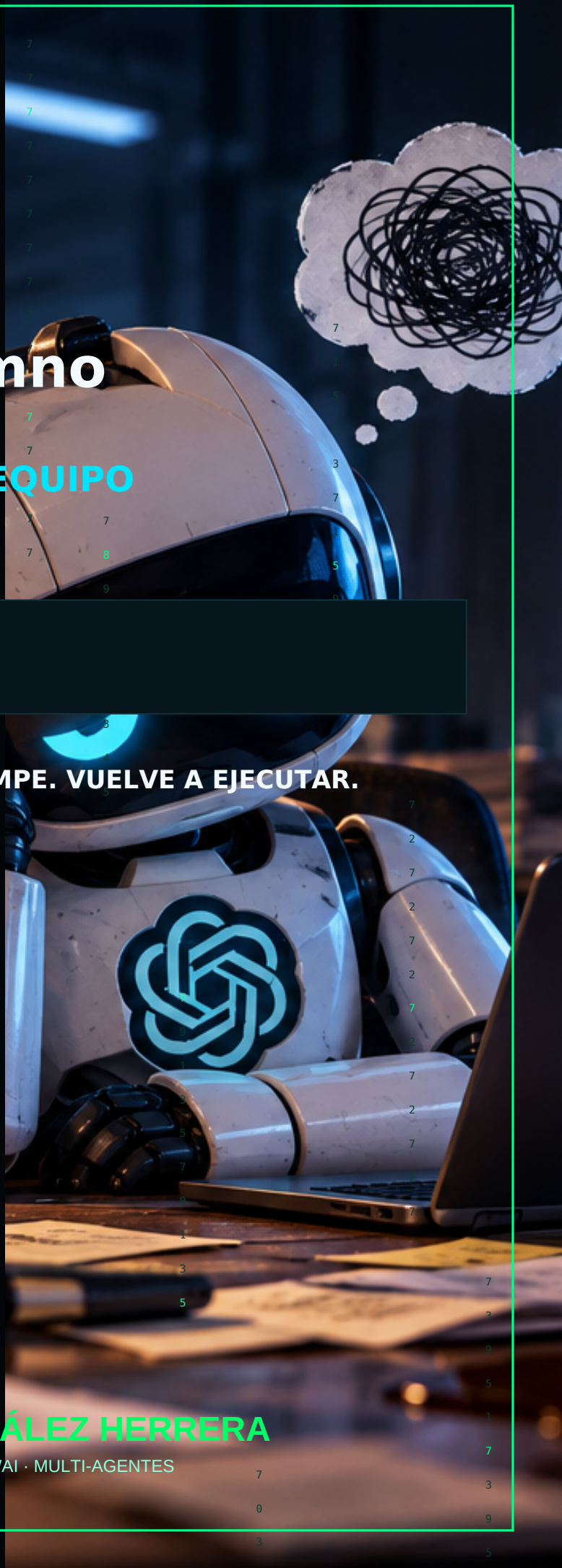
LAS 5 PIEZAS

LLM · AGENT · TASK · CREW · PROCESS

NO COPIES. MODIFICA. ROMPE. VUELVE A EJECUTAR.

SANTIAGO GONZÁLEZ HERRERA

MASTERCLASS · CREWAI · MULTI-AGENTES



01 // EL MAPA

¿Qué estamos construyendo?

Imagina una empresa donde cada persona tiene una función. CrewAI lleva esa idea al mundo de los agentes.

LLM	El cerebro lingüístico
AGENT	El especialista
TASK	El trabajo
CREW	El equipo
PROCESS	La coordinación

MEMORIZA ESTO

Agent = quién · Task = qué · Crew = equipo · Process = cómo · LLM = motor

7
9
1
3
5
7
9
1
3
5
7
9

02 // BOOT

Prepara el sistema

Crea un entorno limpio y comprueba que CrewAI está instalado.

```
python --version
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate

pip install crewai
```

Después comprueba:

```
pip show crewai
```

OBJETIVO

Llegar al siguiente paso con Python, entorno virtual y CrewAI funcionando.

7
9
1
3
5
7
9
1
3
5
7
9

03 // THE BRAIN

Conecta el LLM

CrewAI coordina. El LLM proporciona la capacidad de lenguaje y razonamiento.

```
from crewai import LLM

llm = LLM(
    model="openai/gpt-4o-mini"
)
```

La API key debe estar configurada como variable de entorno.

```
# Windows PowerShell
$env:OPENAI_API_KEY="TU_API_KEY"

# macOS / Linux
export OPENAI_API_KEY="TU_API_KEY"
```

PREGUNTA

¿Qué ocurre si CrewAI está instalado pero el LLM no está correctamente configurado?

LA LÍAS PARDA.

7

9

1

3

5

7

9

1

3

5

7

9

04 // AGENT

Dale una función al agente

Un agente necesita una función clara.

```
from crewai import Agent

investigador = Agent(
    role="Investigador de mercado",
    goal="Encontrar información relevante y fiable",
    backstory="Especialista en tecnología.",
    llm=llm,
    verbose=True
)
```

role → quién es goal → qué quiere conseguir

backstory → contexto llm → motor

AHORA TÚ

Cambia el agente para convertirlo en un experto que te interese.

7
9
1
3
5
7
9
1
3
5
7
9

05 // TASK

Define el trabajo

Una Task convierte el objetivo en una acción concreta y define qué debe entregar.

```
from crewai import Task

investigar = Task(
    description=(
        "Investiga el mercado de impresión 3D."
    ),
    expected_output=(
        "Un resumen con 8 puntos clave."
    ),
    agent=investigador
)
```

REGLA

description = qué debe hacer
expected_output = qué debe entregar

Escribe ahora tu propia Task:

```
mi_task = Task(
    description="_____",
    expected_output="_____",
    agent=_____
)
```

7

9

1

3

5

7

9

1

3

5

7

9

06 // CONTEXT

Pásale el trabajo a otro agente

Aquí aparece la magia del multi-agente: la salida de una tarea alimenta a la siguiente.

```
analista = Agent(  
    role="Analista",  
    goal="Convertir información en conclusiones útiles",  
    backstory="Especialista en análisis.",  
    llm=llm,  
    verbose=True  
)  
  
analizar = Task(  
    description="Analiza la investigación anterior.",  
    expected_output="Conclusiones y oportunidades.",  
    agent=analista,  
    context=[investigar]  
)
```

La conexión importante es:

INVESTIGADOR → RESULTADO → ANALISTA

CONTEXT=[investigar]

Le dice a CrewAI que el segundo trabajo puede utilizar el resultado del primero.

7

9

1

3

5

7

9

1

3

5

7

9

07 // CREW

Monta el equipo

Ahora juntamos agentes, tareas y estrategia de ejecución.

```
from crewai import Crew, Process
```

```
crew = Crew(  
    agents=[investigador, analista],  
    tasks=[investigar, analizar],  
    process=Process.sequential,  
    verbose=True  
)
```

El proceso sequential ejecuta las tareas en orden.

TU TURNO

Añade un tercer agente y piensa qué tarea podría realizar después del analista.

7
9
1
3
5
7
9
1
3
5
7
9

08 // KICKOFF

Ahora dale al botón

```
resultado = crew.kickoff()
```

```
print(resultado)
```

kickoff() inicia la ejecución de la Crew.

OBSERVA EL TERMINAL

¿Qué agente trabaja primero? ¿Qué tarea ejecuta? ¿Qué información recibe el siguiente?

EJECUTA → OBSERVA → MODIFICA → EJECUTA OTRA VEZ

7
9
1
3
5
7
9
1
3
5
7
9

09 // BUILD

Tu Crew completa

```

from crewai import Agent, Task, Crew, Process, LLM

llm = LLM(model="openai/gpt-4o-mini")

investigador = Agent(
    role="Investigador",
    goal="Recopilar información relevante",
    backstory="Especialista en investigación.",
    llm=llm, verbose=True
)

analista = Agent(
    role="Analista",
    goal="Encontrar patrones y conclusiones",
    backstory="Especialista en análisis.",
    llm=llm, verbose=True
)

investigar = Task(
    description="Investiga tendencias de agentes de IA.",
    expected_output="8 puntos clave.",
    agent=investigador
)

analizar = Task(
    description="Analiza la investigación.",
    expected_output="5 conclusiones y 3 oportunidades.",
    agent=analista,
    context=[investigar]
)

crew = Crew(
    agents=[investigador, analista],
    tasks=[investigar, analizar],
    process=Process.sequential,
    verbose=True
)

resultado = crew.kickoff()
print(resultado)

```

7

9

1

3

5

7

9

1

3

5

7

9

10 // FINAL BOSS

Ahora rompe el ejemplo

Diseña una Crew para un problema que conozcas.

LLM

—

AGENT 1: _____
TASK 1: _____

—

AGENT 2: _____
TASK 2: _____

—

AGENT 3: _____
TASK 3: _____

PROCESS: _____

RESULTADO: _____

CONDICIÓN

Como mínimo, una Task debe utilizar el resultado de otra mediante context=[...].

7
9
1
3
5
7
9
1
3
5
7
9

11 // LEVEL UP

Cuando esto ya funciona...

Puedes ampliar el sistema con herramientas y flujos más avanzados.

```
investigador = Agent(  
    role="Investigador",  
    goal="Buscar y contrastar información",  
    backstory="Investigador técnico.",  
    tools=[mi_herramienta],  
    llm=llm,  
    verbose=True  
)
```

Siguiente nivel:

TOOLS · MEMORY · KNOWLEDGE · FLOWS · HIERARCHICAL

NO CORRAS

Primero domina Agent → Task → Crew → Process. Después añade complejidad.

7

9

1

3

5

7

9

1

3

5

7

9

12 // CHEAT SHEET

La chuleta

AGENT
role → quién
goal → objetivo
backstory → contexto
llm → motor
tools → capacidades

TASK
description → qué hace
expected_output → qué entrega
agent → quién la ejecuta
context → qué recibe

CREW
agents → equipo
tasks → trabajos
process → coordinación

RUN
crew.kickoff() → ejecutar

DIVIDE → ASIGNA → CONECTA → EJECUTA → VALIDA

7
9
1
3
5
7
9
1
3
5
7
9

13 // NEXT

Sigue explorando

La sintaxis de CrewAI evoluciona. Consulta la documentación oficial cuando trabajes con nuevas versiones.

CREWAI

Repositorio oficial y documentación del proyecto.

<https://github.com/crewAIInc/crewAI>

Tasks

<https://github.com/crewAIInc/crewAI/blob/main/docs/edge/en/concepts/tasks.mdx>

Crews / Processes

<https://github.com/crewAIInc/crewAI/blob/main/docs/v1.12.2/en/concepts/crews.mdx>

RECURSO DEL CURSO // GITHUB

RECURSO DEL CURSO // GITHUB



COPIA Y PEGA:

<https://github.com/Pendatkill/M-ster-class-.CrewAi-aprender-a-crear-tus-agentes>.

CIERRE DE LA MASTERCLASS

«Como dice mi mujer, en referencia a Los Simpsons:
“todo cal calass señor”.»

Este curso es todo IA, amigos. TODO IA.

Aprende y comparte ■

PARTE 2 · CREACIÓN DE AGENTES

DEL MODELO AL AGENTE CON CAPACIDADES

Pasamos de usar CrewAI como estructura a construir un agente que pueda razonar, ejecutar tarea y utilizar herramientas externas.

EL CAMBIO DE MENTALIDAD

LLM → lenguaje y razonamiento

AGENTE → rol, objetivo, contexto y comportamiento

HERRAMIENTA → capacidad de actuar sobre el exterior

TASK → trabajo ejecutable

CREW → coordinación de agentes y tareas

OBJETIVO: construir un agente investigador y darle una herramienta DDGS para buscar información en Internet.

1 • CONECTAR EL LLM

EL CEREBRO DEL AGENTE

El LLM no es el agente. Es el motor de lenguaje que el agente utiliza para interpretar instrucciones, razonar y decidir qué hacer. El agente se ejecuta dentro de una Task y una Crew.

```
import os
from crewai import LLM, Agent, Task, Crew, Process

os.environ["OPENAI_API_KEY"] = "TU_API_KEY"

llm = LLM(model="openai/gpt-4o-mini")

agent = Agent(
    role="Investigador IA",
    goal="Investigar temas técnicos y explicarlos con claridad",
    backstory="Investigador técnico preciso y curioso.",
    llm=llm,
    verbose=True
)

task = Task(
    description="Explica qué es un agente de IA.",
    expected_output="Una explicación clara y breve.",
    agent=agent
)

crew = Crew(
    agents=[agent],
    tasks=[task],
    process=Process.sequential,
    verbose=True
)
```

CORRECCIÓN IMPORTANTE

El método kickoff() se ejecuta sobre la Crew, no sobre el Agent.

2 · AGENTE + TASK + CREW

PRIMER SISTEMA MULTI-AGENTE

Un agente aislado es perfecto para aprender. Para un sistema reproducible, CrewAI separa responsabilidades: Agent, Task y Crew.

```
from crewai import Agent, Task, Crew, Process

researcher = Agent(
    role="Investigador tecnológico",
    goal="Encontrar información sobre {topic}",
    backstory="Especialista en investigación técnica.",
    llm=llm, verbose=True
)

task = Task(
    description="Investiga {topic} y resume los puntos clave.",
    expected_output="Resumen técnico con 5 puntos.",
    agent=researcher
)

crew = Crew(
    agents=[researcher], tasks=[task],
    process=Process.sequential, verbose=True
)

result = crew.kickoff(inputs={"topic":"agentes de IA"})
print(result.raw)
```

El LLM piensa. El Agent tiene propósito. La Task define el trabajo. La Crew coordina el conjunto.

3 • ¿QUÉ ES DDGS?

AÑADIMOS INFORMACIÓN EXTERNA

DDGS (Dux Distributed Global Search) es una biblioteca de metasearch para Python. Permite buscar texto, noticias, imágenes, vídeos, libros y extraer contenido de URLs.

```
# Instalación
pip install -U ddgs

from ddgs import DDGS

results = DDGS().text(
    "CrewAI agentes de inteligencia artificial",
    max_results=5
)

for r in results:
    print(r["title"])
    print(r["href"])
    print(r["body"])
```

DDGS devuelve datos de búsqueda; el LLM interpreta esos datos. Son capas diferentes.

4 • CONVERTIR DDGS EN UNA TOOL

EL AGENTE YA PUEDE ACTUAR

CrewAI permite crear herramientas personalizadas. Para aprender el concepto, una función decorada con `@tool` es una forma sencilla de exponer DDGS al agente.

```
from crewai.tools import tool
from ddgs import DDGS

@tool("Web Search DDGS")
def web_search(query: str) -> str:
    """Busca información actual usando DDGS."""
    results = DDGS().text(query, max_results=5)

    if not results:
        return "No se encontraron resultados."

    return "\n\n".join(
        f"TÍTULO: {r.get('title', '')}\n"
        f"URL: {r.get('href', '')}\n"
        f"RESUMEN: {r.get('body', '')}"
        for r in results
    )
```

¿Qué ve el LLM? El nombre y la descripción de la tool ayudan al agente a decidir cuándo utilizarla.

5 • CONECTAR DDGS AL AGENTE

DE CHATBOT A AGENTE CON HERRAMIENTAS

```
researcher = Agent(  
    role="Investigador web",  
    goal="Investigar y contrastar información reciente",  
    backstory="Investigador técnico que usa herramientas cuando necesita datos actuales.",  
    llm=llm,  
    tools=[web_search],  
    verbose=True  
)  
  
task = Task(  
    description="Investiga las novedades de {topic}. "  
    "Usa web_search y resume los hallazgos con URLs.",  
    expected_output="Resumen con fuentes y enlaces.",  
    agent=researcher  
)  
  
crew = Crew(  
    agents=[researcher],  
    tasks=[task],  
    process=Process.sequential,  
    verbose=True  
)  
  
result = crew.kickoff(inputs={"topic":"CrewAI"})  
print(result.raw)
```

FLUJO

Usuario → Task → Agent → LLM decide → DDGS → resultados → LLM interpreta → respuesta.

6 • RETO DEL ALUMNO

AHORA TE TOCA CACHARREAR

Modifica el ejemplo para construir un pequeño agente de investigación sobre IA.

- 01 Crea el entorno e instala CrewAI + DDGS.
- 02 Configura el LLM mediante una variable de entorno.
- 03 Crea un Agent con role, goal y backstory.
- 04 Crea una Task que obligue a investigar un tema.
- 05 Conecta la tool Web Search DDGS.
- 06 Haz que el resultado incluya títulos, URLs y resumen.

SIGUIENTE NIVEL

Añade un segundo agente que revise el trabajo del investigador.
Después prueba investigación → análisis → redacción.